Name: JayKishan J Saharan.
Div: D Roll no.: 26
Course/Program: BTech CSE SY
Subject : DSA

DSA Theory assignment: 4.

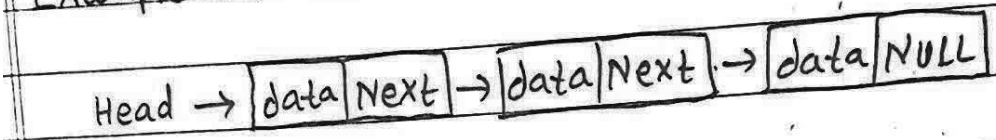
Describe the structure of a node in a singly linked list. Explain how nodes are connected and how data is stored in the list.

A Singly Linked List is a linear data structure composed of a sequence of interconnected nodes. Each node in a Singly Linked List has a distinct structure designed to faciliate this connection and data storage.

Structure of a Node:
A node in a Singly Linked List typically consists of two main components:
Data Part: This field stores the actual value or information that the node represents. The type of data can vary depending on the applications (eg., integers, strings, objects).

Pointer (or Link) Part: This field holds a reference (memory address) to the next node in the sequence. This pointer is crucial for establishing the chain-like structure of the linked list. For the last node in the list, this pointer is typically set to NULL to signify the end of the list.

Example Node Structure:



How Nodes are connected:

- Nodes in a Sigly Linked list are connected sequentially through their next pointers.
- The next pointer of a given node stores the memory address of the subsequent node in the list. This creates a directional link, allowing traversal from the beginning of the list to its end.
- The entire list is managed by a special pointer, often called the head, which points to the very first node in the sequence.
- To access any node within the list, one must start at the head and follow the next pointers until the desired node is reached.

How data is stored:

- Data is stored within the data part of each individual node.
- Unlike in array, where element occupy contigous memory locations, nodes in a linked list can be scattered throughout memory.
- The next pointers provide the logical connection between these physically seperate nodes, maintaining the order of the data. This dynamic allocation of memory allows for efficient insertion and deletion operations without the need to shift large blocks of data.

Q.2) Write Pseudo C++ code to create and display a single Linked List.

Soln:

```
Struct Node {
  int data;
  Node* Next;
  Node (int val) : data (val), next (nullptr) { }
};
  Void insertAtBeginning (Node*& head, int val) {
   Node* newnode = new Node (val);
   newNode -> next = head;
   head = newNode;
}

  Void displayList (Node* head) {
  Node* current = head;
  if (current = ~~head~~ != nullptr) {
   cout << "Linked list is empty";
  }
  return;
  }
  while (current != nullptr) {
  current = current -> next;
  }
}
  int main () {
  Node* head = nullptr;
  insertAtBeginning (head, 30);
  insertAtBeginning (head, 20);
  insertAtBeginning (head, 10);
```

```
displaylist (head);

Node* current = head;
while (current != nullptr) {
 Node* nextnode = current → next;
 delete current;
 current = nextNode;
}
 head = nullptr;
 return 0;
}
```

Q.3) Write Pseudo C++ code to create and display a doubly linked list.

Soln :-

```
struct Node {
 int data;
 Node* Prev;
 Node* next;
Node (int value): data(value), prev(nullptr),
                  next(nullptr) {}
};

Void insertAtEnd (Node*& head, int value) {
Node* newNode = new Node(value);

if (head = nullptr) {
head = new Node;
return;
}
```

```
Node* temp = head;
while (temp -> next != nullptr) {
    temp = temp -> next;
}
    temp -> next = newNode;
    NewNode -> prev = temp;
}

Void displayForward (Node* head) {
    if (head == nullptr) {
    cout << "Linked list is empty";
     return;
    }
    Node* temp = head;
    while (temp != nullptr) {
    if (temp -> next != nullptr) {
     // output: " <-> "
    }
    temp = temp -> next;
    }
}

Void displayBackward (Node* head) {
    if (head == nullptr) {
     // output: "list is empty";
     return;
    }
    Node* temp = head;
    while (temp -> next != nullptr) {
    temp = temp -> next;
}
```

```
while (temp != nullptr) {
 if (temp->prev != nullptr) {
  // output: "<->"
 }
  temp = temp->prev;
}
 //output: "\n"
}

int main() {
 Node* head = nullptr;
 insertAtEnd(head,10);
 insertAtEnd(head,20);
 insertAtEnd(head,30);
 insertAtEnd(head,40);

 displayforward(head);
 displayBackward(head);

 return 0;
}
```

Write Pseudo code c++ to search and delete a node from a singly linked list.

```
struct Node {
 int data;
 Node* next;
};
```

```
void deleteNode (Node*& head, int valueToDelete) {
 if (head == nullptr) {
 cout << "Linked List is empty";
}
  if (head -> data == valueToDelete) {
  Node* temp = head;
  head = head -> next;
  delete temp;
  return;
}
  Node* current = head;
  Node* Previous = nullptr;

  while (current != nullptr && current -> data != valueToDelete
{
  Previous = current;
  current = current -> next;
}
  if (current == nullptr) {
  return;
}
  Previous -> next = current -> next;
 delete current;
}
  int main() {
  Node* head = nullptr;

// Add some nodes
// head = new Node {10, nullptr};
// head -> next = new Node {20, nullptr};
```

```
// head->next->next = new Node {30, nullptr};
// deleteNode(head, 20);
// deleteNode(head, 10);
// deleteNode(head, 50);

return 0;
}
```

q.5) Write Pseudo C++ code to print the elements of a Singly Linked list in reverse order.

Sol^n :-

```
struct Node {
int data;
Node* next;
};
void PrintReverse(Node* head) {
if (head == nullptr) {
cout << "Linked list is empty";
}
return;
PrintReverse(head->next);
cout << head->data << " ";
}
int main() {
Node *head = new Node {10, nullptr};
head->next = new Node {20, nullptr};
head->next->next = new Node {30, nullptr};
cout << "list in reverse order: ";
PrintReverse(head);
return 0;
}
```

Q.6) Write Pseudo C++ code to sort a Signly Linked List in ascending order.

Sol^n:-

```
struct Node {
 int data;
 Node* next;
};
 Void SortLists (Node* head) {
 if (head==nullptr){
  cout<<"linked list is empty";
 }
  return;
 Node* i;
 Node* j;
 int temp;

 for (i=head ; i->next != nullptr ; i=i->next ) {
  for (j=i->next ; j != nullptr ; j=j->next){
   if (i->data > j->data){
    temp=i->data;
    i->data=j->data;
    j->data=temp;
   }
  }
 }
}
 int main (){
 Node* head=new Node {30, nullptr};
 head->next=new Node {10, nullptr};
 head->next->next=new Node {20, nullptr};
```

```
cout << "Before Sorting: 3 -> 2 -> 2 -> ";
sortLists(head);

cout << "After Sorting: ";
Node* temp = head;
while (temp != nullptr) {
    cout << temp->data << " ";
    temp = temp->next;
}

return 0;
}
```

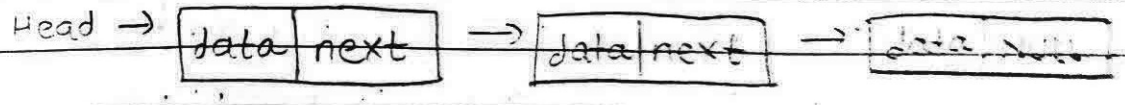
Q.7) Compare Singly Linked List and doubly Linked List based on the following:

1. Structure of the node.
2. Insertions and deletion operations.
3. Traversal capabilities.

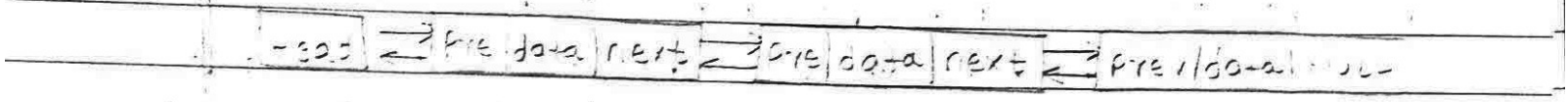
Sol^n:-

1. Structure of the node

- Singly Linked List: Each node consists of two fields: a data field to store the value and a single pointer (often called next) that holds the memory address of the subsequent node in the sequence. The last node's next pointer is null to signify the end of the list. This design is memory-efficient because each node only requires a single pointer.



- Doubly Linked List: Each node contains three fields: a data field, a next pointer pointing to the next node and an additional previous pointer pointing to the preceding node. This extra pointer allows for bidirectional linking but increases memory comsumption per node.



2. Insertion and deletion operation

- Singly Linked List: Insertion and deletion are efficient at the beginning of the list (O(1) time complexity). However, deleting a node from the middle or end of the list requires traversing the list from beginning to find the preceding node, making the operation less efficient (O(n) time complexity).

- Doubly Linked List: Insertion and deletion are more efficient. The presence of the previous pointer allows allows for these operations to be performed in O(1) time if you have a reference to the node you want to delete. This is because you can easily update the pointers of the adjacent nodes without needing to traverse the entire list to find the previous node.

3. Traverse capabilities.

- Singly Linked List: Traversal can only be done in one direction: forward, from the head of the list to tail.

- Doubly Linked List: Traversal can be performed in both directions: forward and backward, due to the previous and next pointers in each node. This makes it useful for applications like undo/redo functionality or web browser history.

Q.8) Which is better for a program that needs to move forwards and backwards through data: a singly Linked List or a doubly Linked List? Explain your choice with reasons.

Sol^n: for a program that needs to move both forward and backward through data, a doubly Linked List is better choice.

Reasons:

- Bidirectional Traversal: Each node contains two pointers: one pointing to the next node (next) and another pointing to the previous node (prev). This structure inherently supports traversal in both directions with equal efficiency.

- Efficient backward movement: when a program requires frequent backward navigation, the prev pointer in a doubly Linked List allows for direct access to the preseding element in O(1) time.